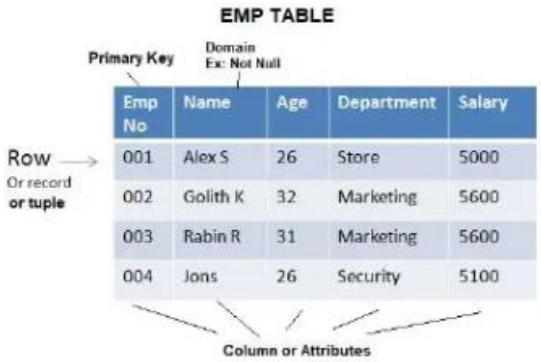
UNIT II

Relational Model: Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus.
BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

RELATIONAL DATA MODEL

- Relational model is the **most popular model** and the most extensively used model.
- The relational model proposed by CODD.
- The relational model represents the database as a collection of **relations(tables).** Every row in the table represents a collection of related data values. These rows in the table denote a real-world entity or relationship.

The table name and column names are helpful to interpret the meaning of values in each row. The data are represented as a set of relations. In the relational model, data are stored as tables. However, the physical storage of the data is independent of the way the data are logically organized.



Number of rows is called Cardinality. Example above table has Cardinality 4.
Number of columns is called Degree. Example above table has degree 5.

Key Characteristics of the Relational Model:

- Data Representation: Data is organized in tables (relations), with rows (tuples) representing records and columns (attributes) representing data fields.
- Atomic Values: Each attribute in a table contains atomic values, meaning no multi-valued or nested data is allowed in a single cell.
- Unique Keys: Every table has a primary key to uniquely identify each record, ensuring no duplicate rows.
- Attribute Domain: Each attribute has a defined domain, specifying the valid data types and constraints for the values it can hold.

- Tuples as Rows: Rows in a table, called tuples, represent individual records or instances of real-world entities or relationships.
- Relation Schema: A table's structure is defined by its schema, which specifies the table name, attributes, and their domains.
- Data Independence: The model ensures logical and physical data independence, allowing changes in the database schema without affecting the application layer.
- Integrity Constraints: The model enforces rules like:
    - Domain constraints: Attribute values must match the specified domain.
    - Entity integrity: No primary key can have NULL values.
    - Referential integrity: Foreign keys must match primary keys in the referenced table.
- Relational Operations: Supports operations like selection, projection, join, union, and intersection, enabling powerful data retrieval manipulation.
- Data Consistency: Ensures data consistency through constraints, reducing redundancy and anomalies.

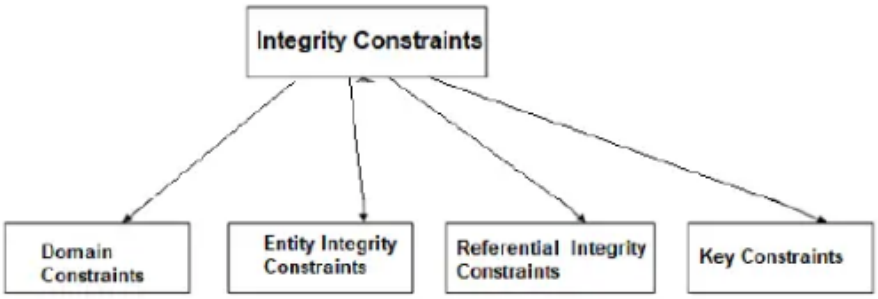
Domain

- Columns in table have a unique name, often referred as attributes in DBMS. A domain is a unique set of values permitted for an attribute in a table. For example, a domain of month-of-year can accept January, February….December as possible values, a domain of integers can accept whole numbers that are negative, positive and zero.
- Domain: It contains a set of atomic values that an attribute can take.

Relational Integrity Constraints

- While designing Relational Model, we define some conditions which must hold for data present in database are called Constraints.
- These constraints are checked before performing any operation (insertion, deletion and updation) in database.
    - If there is a violation in any of constrains, operation will fail.

Constraints on the Relational database management system is mostly divided into 4 categories are:



1. Domain Constraints
2. Entity Integrity Constraints
3. Key Constraints
4. Referential Integrity Constraints

Domain Constraints

Domain constraints can be violated if an attribute value is not appearing in the corresponding domain or it is not of the appropriate data type.

Domain constraints specify that within each tuple, and the value of each attribute must be unique. This is specified as data types which include standard data types integers, real numbers, characters, Booleans, variable length strings, etc.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1004	Morgan	8th	A

Not allowed. Because AGE is an integer attribute

Domain Constraints are Data type constraints, length constraints, Not null constraints…

Entity integrity constraints

- The entity integrity constraint states that primary key value can't be null.
- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

EXAMPLE:

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

Key Constraints

An attribute that can uniquely identify a tuple in a relation is called the key of the table. The value of the attribute for different tuples in the relation has to be unique.

Why Key Constraints Are Important?

- Prevent Duplicates: Ensure unique identification of rows.
- Maintain Relationships: Enable proper linking between tables (via foreign keys).
- Enforce Data Integrity: Prevent invalid or inconsistent data.

Primary Key Constraints

- A "primary key constraint" is a specific type of "key constraint" in a database,
- The PRIMARY KEY constraint uniquely identifies each record in a table.
- Primary keys must contain UNIQUE values, and cannot contain NULL values.
- A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields).
- Primary keys can be classified into two categories Simple primary key that consists of one column and composite primary key that consists of Multiple column.
- Primary keys can be defined in CREATE TABLE or ALTER TABLE:

Example:

In the given table, CustomerID is a primary key attribute of Customer Table. It is most likely to have a single key for one customer, CustomerID =1 is only for the CustomerName =" Google".

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive

Unique Key Constraints

A Unique Key ensures that the values in a column are unique, but unlike a primary key, it allows one NULL value.

Employee_ID	Email	Name
1	john@gmail.com	Aniket Kumar
2	NULL	Shashwat Dubey
3	stalin@ gmail.com	Manvendra Sharma

Unique Values: The email column must contain unique values.
john@gmail.com and stalin@ gmail.com are valid.
Adding another row with john@gmail.com would result in an error.
Allows One NULL: The email column can contain one NULL value.

Valid: NULL in the second row.
Invalid: Adding another row with NULL in email will be rejected.

Referential Integrity Constraints

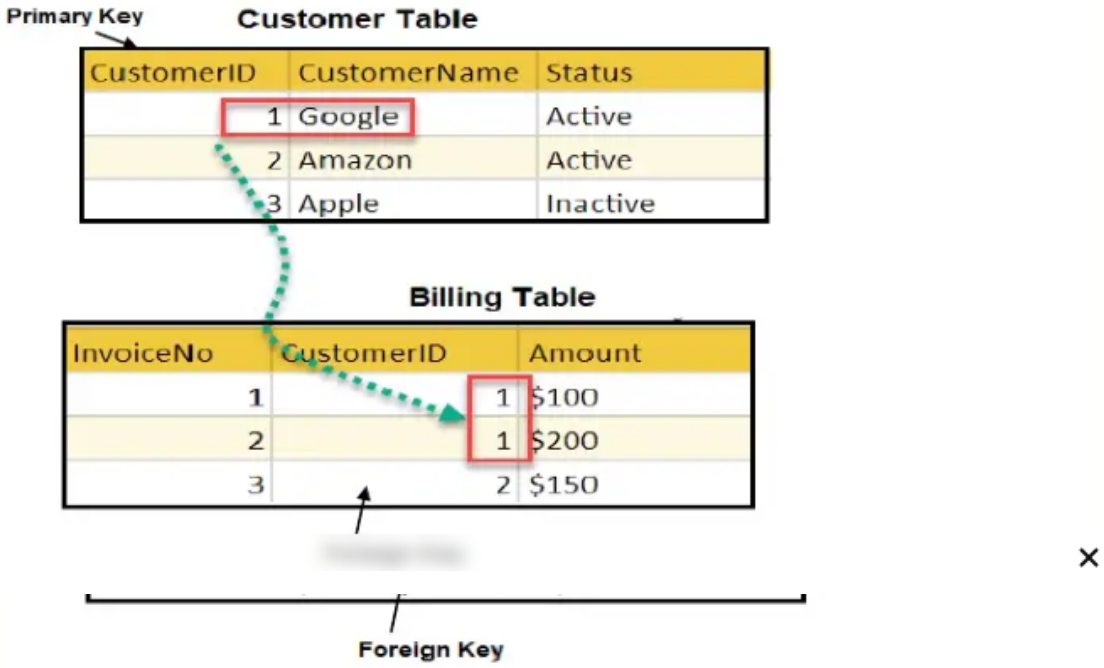
- Referential integrity constraints are rules that ensure relationships between tables remain consistent.

- They enforce that a foreign key in one table must either match a value in the referenced primary key of another table or be NULL.
- This guarantees the logical connection between related tables in a relational database.
- Referential Integrity constraints in DBMS are based on the concept of Foreign Keys.

Why Referential Integrity Constraints Are Important ?

- Maintains Consistency: Ensures relationships between tables are valid.
- Prevents Orphan Records: Avoids cases where a record in a child table references a non-existent parent record.
- Enforces Logical Relationships: Strengthens the logical structure of a relational database.

In the example, we have 2 relations, Customer and Billing.
Tuple for CustomerID =1 is referenced twice in the relation Billing. So we know CustomerName=Google has billing amount $300



IMPORTANCE OF NULL VALUES

In SQL there may be some records in a table that do not have values for every field and those fields are termed as a NULL value.

The scenarios where null values are used:
Null values can be used in cases of

- Missing data
- Unknown data
- Not applicable data
- When a mathematical or statistical computation can't be performed.

NULL values could be possible because at the time of data entry information is not available. So SQL supports a special value known as NULL which is used to represent the values of attributes that may be unknown or not apply to a tuple.
SQL places a NULL value in the field in the absence of a user-defined value.
For example, the Apartment_number attribute of an address applies only to addresses that are in apartment buildings and not to other types of residences.
So, **NULL** values are those values in which there is no data value in the particular field in the table.

Importance of NULL Value

- It is important to understand that a NULL value differs from a zero value.
- A NULL value is used to represent a missing value, but it usually has one of three different interpretations:
    - The value unknown (value exists but is not known)
    - Value not available (exists but is purposely withheld)
    - Attribute not applicable (undefined for this tuple)
- It is often not possible to determine which of the meanings is intended. Hence, SQL does not distinguish between the different meanings of NULL.

Principles of NULL values

- Setting a NULL value is appropriate when the actual value is unknown, or when a value is not meaningful.
- A NULL value is not equivalent to a value of ZERO if the data type is a number and is not equivalent to spaces if the data type is a character.
- A NULL value can be inserted into columns of any data type.
- A NULL value will evaluate NULL in any expression.
- Suppose if any column has a NULL value, then UNIQUE, FOREIGN key, and CHECK constraints will ignore by SQL.

RELATIONAL ALGEBRA

Relational algebra is a procedural query language, which takes instances of relations as input and yields instances of relations as output. It uses operators to perform queries. An operator can be either unary or binary. They accept relations as their input and yield relations as their output. Relational algebra is performed recursively on a relation and intermediate results are also considered relations.

The fundamental operations of relational algebra are as follows −

- Select
- Project
- Union
- Set-different
- Cartesian product
- Join
- Rename

Project Operation (∏)

It projects column(s) that satisfy a given predicate.

Notation − ∏A1, A2, An (r)

Where A1, A2 , An are attribute names of relation r.

Duplicate rows are automatically eliminated, as relation is a set.

For example −

∏subject, author (Books)

Selects and projects columns named as subject and author from the relation Books..

Union Operation (∪)

It performs binary union between two given relations and is defined as −

$r \cup s = \{ t | t \in r or t \in s\}$

Notation − r U s

Where r and s are either database relations or relation result set (temporary relation).

For a union operation to be valid, the following conditions must hold −

- r, and s must have the same number of attributes.
- Attribute domains must be compatible.

- Duplicate tuples are automatically eliminated.

Π author (Books) ∪ Π author (Articles)

Output − Projects the names of the authors who have either written a book or an article or both.

Intersection Operation (∩)

It displays the common values in r & s. It is denoted by ∩.

r ∩ s

Example:

Π author (Books) ∩ Π author (Articles)

Output − Projects the names of the authors who have written both book and an article.

Set Difference (−)

The result of set difference operation is tuples, which are present in one relation but are not in the second relation.

Notation − r − s

Finds all the tuples that are present in r but not in s.

Π author (Books) − Π author (Articles)

Output − Provides the name of authors who have written books but not articles.

Cartesian Product (X)

Combines information of two different relations into one.

On applying CARTESIAN PRODUCT on two relations that is on two sets of tuples, it will take every tuple one by one from the left set(relation) and will pair it up with all the tuples in the right set(relation).

So, the CROSS PRODUCT of two relation A(R1, R2, R3, …, Rp) with degree p, and B(S1, S2, S3, …, Sn) with degree n, is a relation C(R1, R2, R3, …, Rp, S1, S2, S3, …, Sn) with degree p + n attributes.

The cardinality (number of tuples) of resulting relation from a Cross Product operation is equal to the number of attributes(say m) in the first relation multiplied by the number of attributes in the second relation(say n).

Cardinality = m*n

Notation:

r X s

Example:

Consider two relations STUDENT(SNO, FNAME, LNAME) and DETAIL(ROLLNO, AGE) below:

STUDENT table:

SNO	FNAME	LNAME
1	Albert	Smith
2	Pearl	Weber

DETAIL table:

ROLLNO	AGE
5	18
9	21

On applying CROSS PRODUCT on STUDENT and DETAIL,

STUDENT × DETAILS

we get:

SNO	FNAME	LNAME	ROLLNO	AGE
1	Albert	Smith	5	18
1	Albert	Smith	9	21
2	Pearl	Weber	5	18
2	Pearl	Weber	9	21

In general, we don't use cartesian Product unnecessarily, which means without proper meaning we don't use Cartesian Product. Generally, we use Cartesian Product followed by a Selection operation and comparison on the operators as shown below :

$\sigma_{A=D}(A \times B)$

Example: $\sigma_{\text{author = 'Raghu Ramakrishnan'}}$(Books X Articles)

Output – Displays a relation, which shows all the books and articles written by 'Raghu Ramakrishnan'

Join Operations:

A Join operation combines related tuples from different relations, if and only if a given join condition is satisfied. It is denoted by ⋈.

Example:

EMPLOYEE

EMP_CODE	EMP_NAME
101	Stephan
102	Jack
103	Harry

SALARY

EMP_CODE	SALARY
101	50000
102	30000
103	25000

Operation: (EMPLOYEE ⋈ SALARY)

EMP_CODE	EMP_NAME	SALARY
101	Stephan	50000
102	Jack	30000
103	Harry	25000

Example 2: Project EMP_NAME and SALARY from two table based on their EMP_CODE.

$\Pi_{EMPNAME, SALARY}$ (EMPLOYEE ⋈ SALARY)

Output:

EMP_NAME	**SALARY**
Stephan	50000
Jack	30000
Harry	25000

Difference between Join and Cartesian Product in Relational Algebra:

Cartesian Product	**Join**
Generates all possible combinations of rows between two tables.	Combines rows from multiple tables based on a specific matching condition between columns.
No filtering condition is applied and it is only one operation.	Different types of joins exist like inner join, left join, right join, full outer join, each with its own filtering logic.

Rename Operation (ρ)

The results of relational algebra are also relations but without any name. The rename operation allows us to rename the output relation. 'rename' operation is denoted with small Greek letter rho ρ.

Notation – ρ x (E)

Where the result of expression E is saved with name of x.

RELATIONAL CALCULUS

In contrast to Relational Algebra, Relational Calculus is a non-procedural query language, that is, it tells what to do but never explains how to do it.

Relational calculus exists in two forms:

1. Tuple Relational Calculus (TRC)
2. Domain Relational Calculus (DRC)

Tuple Relational Calculus (TRC)

TRC is based on the concept of tuples, which are ordered sets of attribute values that represent a single row or record in a database table.

Filtering variable ranges over tuples.

Notation − {T | Condition}

Where T is a tuple variable and the above returns all tuples T that satisfies a condition.

For example (1)−

{ T.name | Author(T) AND T.article ='database' }

Output − Returns tuples with 'name' from Author who has written article on 'database'.

For example (2)−

{ T.name | Employees(T) ∧ T.Salary >50000 }

Output − Returns tuples with 'name' from Employees whose salary >50000.

TRC can be quantified. We can use Existential (∃) and Universal Quantifiers (∀).

Existential Quantifier: ∃T ∈ R(Cond) will succeed if Cond succeeds for at least one tuple in T.

Universal Quantifier: ∀T ∈ R(Cond) will succeed if Cond succeeds for at all tuples in T.

Domain Relational Calculus (DRC)

In DRC, the filtering variable uses the domain of attributes instead of entire tuple values (as done in TRC, mentioned above).

Notation −

{ a1, a2, a3, ..., an | P (a1, a2, a3, ... ,an)}

Where a1, a2 are attributes and P stands for formulae built by inner attributes.

For example −

{<article, page, subject >| ∈ TutorialsPoint ∧ subject ='database'}

Output − Yields Article, Page, and Subject from the relation TutorialsPoint, where subject is database.

Just like TRC, DRC can also be written using existential and universal quantifiers. DRC also involves relational operators.

The expression power of Tuple Relation Calculus and Domain Relation Calculus is equivalent to Relational Algebra.

Select Operation (σ)

It selects tuples that satisfy the given predicate from a relation.
Notation :-

$$\sigma_p(r)$$

Where σ stands for selection predicate and r stands for relation. p is prepositional logic formula which may use connectors like and, or, and not. These terms may use relational operators like − $=, \neq, \geq, <, , >, \leq$.

For example 1:

$$\sigma_{subject = "database"}(Books)$$

Output − Selects tuples from books where subject is 'database'.

For example 2:

$$\sigma_{age >= 40}(Teacher)$$

Output − Selects tuples from Teacher whohse age is greater than or equal to 40.

For example 3:

$$\sigma_{\text{subject} = \text{"database"} \text{ and price} = 450}(Books)$$

Output − Selects tuples from books where subject is 'database' and 'price' is 450.

STRUCTURED QUERY LANGUAGE (SQL)

SQL is a database language designed for the retrieval and management of data in a relational database. **SQL** stands for **Structured Query Language**.
SQL includes database creation, deletion, fetching rows, modifying rows, etc.

Applications of SQL

As mentioned before, SQL is one of the most widely used query language over the databases. Allows users to access data in the relational database management systems.

- Allows users to describe the data.
- Allows users to define the data in a database and manipulate that data.
- Allows to embed within other languages using SQL modules, libraries & pre-compilers.
- Allows users to create and drop databases and tables.
- Allows users to create view, stored procedure, functions in a database.
- Allows users to set permissions on tables, procedures and views.

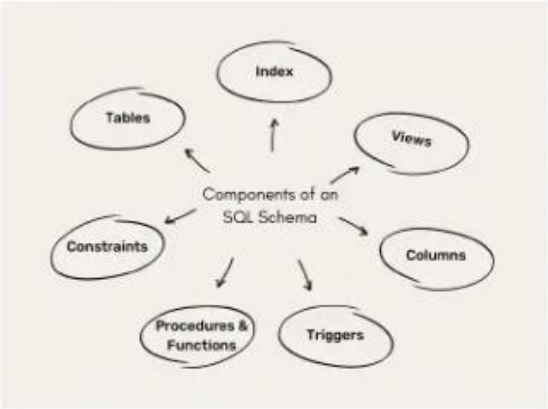
SQL databases:

- Microsoft SQL
- MySQL
- Oracle
- PostgreSQL
- MSSQL

DATABASE SCHEMA

A schema is the blueprint or structure that defines how data is organized and stored in a database. It outlines the tables, fields, relationships, views, indexes, and other elements within the database. The schema defines the logical view of the entire database and specifies the rules that govern the data, including its types, constraints, and relationships.

Components of an SQL Schema:



DATA TYPES

1. **CHAR(Size):** This data type is used to store character string values of fixed length. The maximum number of character is 255 characters.

2. **VARCHAR(Size) / VERCHAR2(Size)**: This data type is used to store variable length string data. The maximum character can hold is 2000 character.

3. **NUMBER(P, S):** The NUMBER data type is used to store number (fixed or floating point). P is precision and S is scale.
   - The precision is the number of digits in a number. It ranges from 1 to 38.
   - The scale is the number of digits to the right of the decimal point in a number. It ranges from -84 to 127.

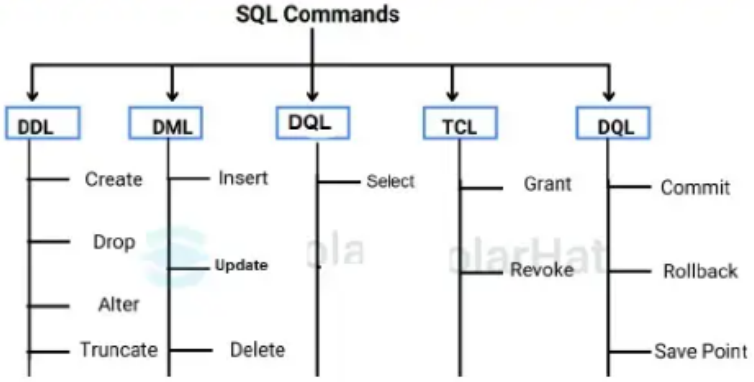
For example, the number 1234.56 has a precision of 6 and a scale of 2. So to store this number, you need NUMBER(6,2).
P and S are optional.

4. **INT**(size) or INTEGER(size) , **FLOAT**, **DOUBLE**, DECIMAL are also used
5. **DATE:** This data type is used to represent date and time. The standard format is DD-MMM- YYYY as in 17-SEP-2009.
6. **RAW:** The RAW data type is used to store binary data, such as digitized picture or image.
7. **BOOL or BOOLEAN** : True or false

SQL COMMAND CATEGORIES:

SQL has the following categories of commands used for managing the databases

1. Data Definition Language (DDL)
2. Data Manipulation Language (DML)
3. Data Query Language (DQL)
4. Transational Control Language (TCL)
5. Data Control Language (DCL)



DDL(Data Definition Language)

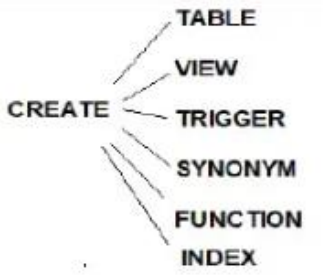
DDL commands are used to work on database objects like TABLE, VIEW, SYNONYM, etc.
DDL Commands : Create, Alter, Drop, Truncate, Rename

CREATE COMMAND:

CREATE is used to create the database or its objects (like table, index, function, views, store procedure and triggers).

CREATE COMMAND:

CREATE is used to create the database or its objects (like table, index, function, views, store procedure and triggers).



CREATE TABLE command is used to create structure of a Table.

Syntax of **create table** command:

```
CREATE TABLE tablename
(  column1  datatype(size),
   column2  datatype(size),
   column3  datatype(size),
   .....
  columnN datatype(size)
);
```

CREATE and TABLE are the keywords.
Example 1:
Create a PERSON table with (name,city and state columns)

```
SQL>Create table person
    (
     name varchar2(10),
     city varchar2(10),
     state varchar2(10)
    );
```

Example2: create student table with (rollno, name, branch , mobile)

```
SQL>Create table student
    (
     rollno   varchar2(10),
     name     varchar2(10),
     branch varchar2(10),
     mobile number(10)
    );
```

ALTER COMMAND:

ALTER TABLE is used to alter the structure of the database by add Columns, modify size and delete an empty column

Syntax1: To add new columns

Alter table tablename add (column datatype(size), ...);

Example: Query to add new column 'PINCODE' to the STUDENT table

SQL>ALTER TABLE STUDENT ADD (PINCODE NUMBER (10));

Syntax2: To modify size(increase only) or change data type

Alter table tablename modify (column datatype(size));

Example: Query to modify size of the NAME attribute in the table 'STUDENT'

SQL>ALTER TABLE STUDENT MODIFY (NAME VARCHAR2 (15));

Syntax3: To delete an empty column

Alter table *tablename* drop column *columnname*;

Example: Query to delete 'pincode' column

SQL>ALTER TABLE STUDENT DROP COLUMN PINCODE;

Syntax4: To rename a column

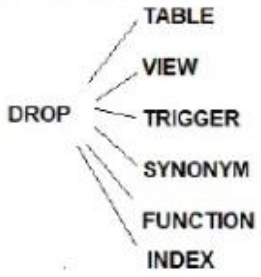
Alter table *tablename* rename column oldcolumn_name **to** Newcolumn_*name*;

Example: Query to change attribute name from NAME to STNAME

SQL>ALTER TABLE STUDENT RENAME COLUMN *NAME* TO *STNAME*;

DROP COMMAND:

DROP – is used to delete objects from the database. It is DDL Command.



DROP TABLE:

A DROP TABLE statement is used to delete a table structure and all data from a table. We cannot rollback.

Syntax: DROP TABLE tableName;
Example: SQL> DROP TABLE student;

TRUNCATE COMMAND:

TRUNCATE TABLE – is used to delete all data from a table. It is a DDL command.

.

Syntax: TRUNCATE TABLE tableName;
Example: SQL> TRUNCATE TABLE student;

Difference between DROP and TRUNCATE in SQL

S.No	DROP	TRUNCATE
1	It is used to eliminate the whole database from the table.	It is used to eliminate the tuples from the table.
2	Integrity constraints get removed in the DROP command.	Integrity constraint doesn't get removed in the Truncate command.
3	The structure of the table does not exist.	The structure of the table exists.

DML (Data Manipulation Language)

These commands that deals with the manipulation of data present in the database.
DML commands are **INSERT, UPDATE , DELETE**

INSERT COMMAND:

INSERT is used insert the data into the table. It is DML command.
Syntax:

INSERT into tablename(Column1, Column2..) values (value1,value2,…);

Column1, Column2.. : Specify the column names only when to insert data for specific column values for the record.

Example:
Create table with table name "teacher" and insert records and display the records.
SQL> create table teacher (id number(3), name varchar2(10), age number(3));
SQL> insert into teacher values(1, 'A.RAO', 30);
SQL> insert into teacher (id , name) values(2, 'B.RAO');
Inserted row with teacher id and name only.

INSERT values in interactive input mode using & symbol.

Example: SQL>insert into teacher values(&id, '&name', &age);

UPDATE COMMAND:

UPDATE command is used to set the new data to the columns. It is DML command.
Syntax:
UPDATE tablename SET Column1=new value, Column2=new value .. where condition;

Example: Change the age of teacher to 45 whose id=3;
SQL> UPDATE teacher SET age=45 where id=3;

Example: Change the name of teacher to 'ARJUN RAO' whose name is 'A.RAO'
SQL> UPDATE teacher SET name='ARJUN RAO' where name='A.RAO';

DELETE COMMAND:

DELETE command is used to delete a specific single record or group of records or all records based on the given condition in WHERE clause. It is DML command.
It delete the records temporarily from the table. We can rollback the deleted records before COMMIT command.

Syntax:
DELETE from tablename where condition;
Example: **Delete a teacher record whose id=3;**
SQL> DELETE from teacher where id=3;
Example: Delete a teacher record whose name is 'B.RAO'
SQL>DELETE from teacher where name='B.RAO';

Example: Delete a teacher record whose age is 60
SQL>DELETE from teacher where age=60;

DQL (Data Query Language)

DQL command is SELECT that is used to display records of the table.
Syntax for simple SELECT:

SELECT Column_Name1, Column_Name2, ..., Column_NameN FROM Table_Name;

In this SELECT syntax, Column_Name1, Column_Name2, ….., Column_NameN are the name of those columns in the table whose data we want to read.

If you want to access all rows from all fields of the table, use the following SELECT syntax with * asterisk sign:

```
SELECT * FROM Table_Name;
```

Example: Query to display all records of a table 'STUDENT'

```
SQL>SELECT * FROM STUDENT;
```

Example: Query to display only ROLLNO & STNAME columns of a table 'STUDENT'

```
SQL>SELECT ROLLNO, NAME FROM STUDENT;
```

DCL (Data Control Language)

DCL includes commands deal with the rights, permissions and other controls of the database system. These are GRANT and REVOKE commands.

GRANT-gives user's access privileges to the database.
Example: Query to grant delete permission to manager on Bank table.

```
SQL>GRANT DELETE ON BANK TO MANAGER;
```

REVOKE-withdraw user's access privileges given by using the GRANT command.

```
SQL>REVOKE DELETE ON BANK FROM MANAGER;
```

TCL (Transaction Control Language)

TCL commands deal with the transaction within the database.

COMMIT– commits a Transaction.
Example: SQL>COMMIT;

ROLLBACK– rollbacks a transaction in case of any error occurs.
Example: SQL>ROLLBACK;

SAVEPOINT–sets a savepoint within a transaction.
Example: SQL>SAVEPOINT A;
The above set a label 'A' for transaction at particular point.

Suppose if we want rollback till savepoint A:

```
SQL>ROLLBACK TO SAVE POINT A;
```

Where A is savepoint label.